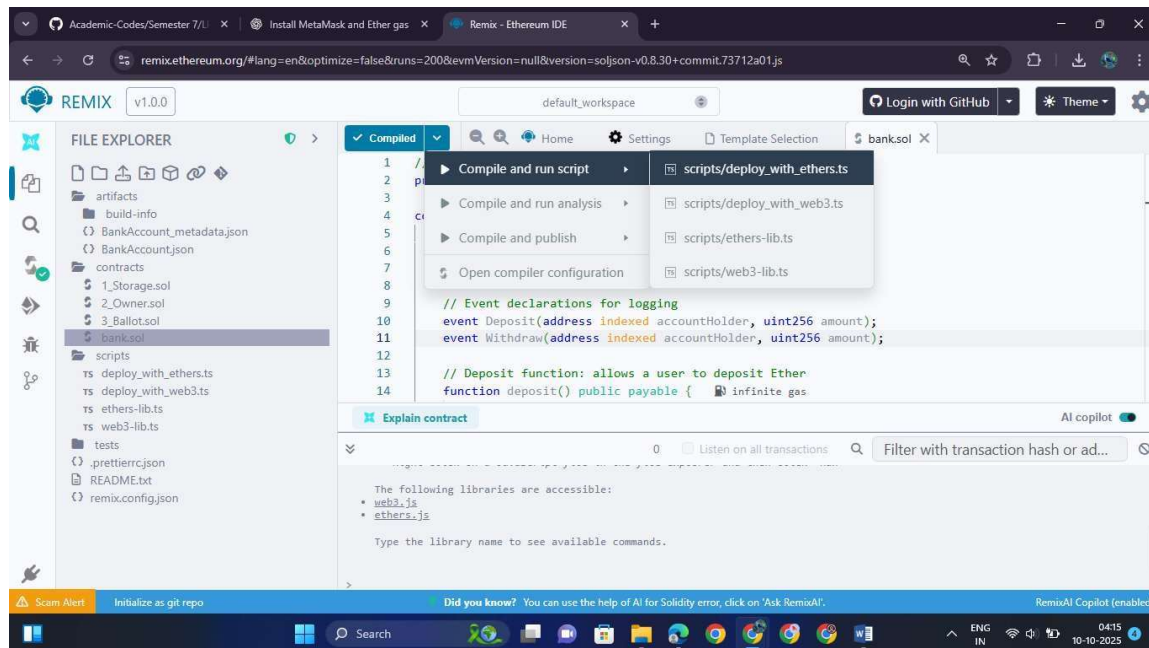


Practical 3: Write a smart contract on a test network, for Bank account of a customer for following operations:

- Deposit money
- Withdraw Money
- Show balance



```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract BankAccount {
```

```
    // Mapping to store balance of each account holder
```

```
    mapping(address => uint256) private balances;
```

```
    // Event declarations for logging
```

```
    event Deposit(address indexed accountHolder, uint256 amount);
```

```
    event Withdraw(address indexed accountHolder, uint256 amount);
```

```
    // Deposit function: allows a user to deposit Ether
```

```
    function deposit() public payable {
```

```
        require(msg.value > 0, "Deposit amount must be greater than zero");
```

```
        balances[msg.sender] += msg.value;
```

```
        emit Deposit(msg.sender, msg.value);
```

```
}
```

```

// Withdraw function: allows a user to withdraw Ether
function withdraw(uint256 _amount) public {
    require(_amount > 0, "Withdrawal amount must be greater than zero");
    require(balances[msg.sender] >= _amount, "Insufficient balance");
    balances[msg.sender] -= _amount;
    // Transfer Ether to the caller
    payable(msg.sender).transfer(_amount);
    emit Withdraw(msg.sender, _amount);
}

// Show balance function: returns the balance of the caller
function showBalance() public view returns (uint256) {
    return balances[msg.sender];
}
}

```

Withdraw:

AccountHolder: 0xUserAddress123

Amount: 0.4 ETH

Balance: 1.3 ETH